Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Principle: Count function calls

©Mohammad Abdulaziz

Principle: Count function calls

For every function $f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \tau$

define a *timing function* $t_f :: \tau_1 \Rightarrow \cdots \Rightarrow \tau_n \Rightarrow \text{nat}$:

©Mohammad Abdulaziz

Example

```
app [] ys = ys
```

©Mohammad Abdulaziz

Example

```
app [] ys = ys
```

$\rightsquigarrow$

$$\texttt{t_app [] ys} = \underbrace{1}_{\texttt{ys}} + \underbrace{1}_{\text{call of } \texttt{app}}$$

Example

```
app [] ys = ys
```

$\rightsquigarrow$

$$\texttt{t_app [] ys} = \underbrace{1}_{\texttt{ys}} + \underbrace{1}_{\text{call of } \texttt{app}}$$

```
app (x#xs) ys = x # app xs ys
```

©Mohammad Abdulaziz

Example

```
app [] ys = ys
```

$\rightsquigarrow$

$$\texttt{t_app [] ys} = \underbrace{1}_{\texttt{ys}} + \underbrace{1}_{\text{call of } \texttt{app}}$$

```
app (x#xs) ys = x # app xs ys
```

$\rightsquigarrow$

$$\texttt{t_app (x\#xs) ys} = \underbrace{1}_{\texttt{x}} + (\underbrace{1}_{\texttt{xs}} + \underbrace{1}_{\texttt{ys}} + \underbrace{\texttt{t_app xs ys}}_{\text{recursive call}}) + \underbrace{1}_{\texttt{\#}} + \underbrace{1}_{\text{call of } \texttt{app}}$$

©Mohammad Abdulaziz

A compact formulation of $e \rightsquigarrow t$

©Mohammad Abdulaziz

A compact formulation of $e \rightsquigarrow t$

t is the sum of all $t_g\ s_1 \ldots s_k$
such that $g\ s_1 \ldots s_k$ is a subterm of e

©Mohammad Abdulaziz

A compact formulation of $e \rightsquigarrow t$

t is the sum of all $t_g\ s_1 \ldots s_k$
such that $g\ s_1 \ldots s_k$ is a subterm of e

If g is

- a variable, a constant, a constructor or
- a predefined function on `bool` or numbers

then $t_g \cdots = 1$.

if and *case*

©Mohammad Abdulaziz

if and *case*

So far we model a call-by-value semantics

©Mohammad Abdulaziz

if and *case*

So far we model a call-by-value semantics

Conditionals and case expressions are evaluated lazily.

©Mohammad Abdulaziz

if and *case*

So far we model a call-by-value semantics

Conditionals and case expressions are evaluated lazily.

Translation:

$$\frac{b \rightsquigarrow t \quad s_1 \rightsquigarrow t_1 \quad s_2 \rightsquigarrow t_2}{\textbf{if } b \textbf{ then } s_1 \textbf{ else } s_2 \rightsquigarrow t + (\textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2)}$$

©Mohammad Abdulaziz

if and *case*

So far we model a call-by-value semantics

Conditionals and case expressions are evaluated lazily.

Translation:

$$\frac{b \rightsquigarrow t \quad s_1 \rightsquigarrow t_1 \quad s_2 \rightsquigarrow t_2}{\textbf{if } b \textbf{ then } s_1 \textbf{ else } s_2 \rightsquigarrow t + (\textbf{if } b \textbf{ then } t_1 \textbf{ else } t_2)}$$

Similarly for **case**

©Mohammad Abdulaziz

$O(\cdot)$ is enough

©Mohammad Abdulaziz

$O(\cdot)$ is enough

$\Longrightarrow$ Reduce all additive constants to 1

©Mohammad Abdulaziz

$O(\cdot)$ is enough

$\Longrightarrow$ Reduce all additive constants to 1

Example

```
t_app (x#xs) ys = t_app xs ys + 1
```

$O(\cdot)$ is enough

$\Longrightarrow$ Reduce all additive constants to 1

Example

```
t_app (x#xs) ys = t_app xs ys + 1
```

$\Longrightarrow$ Count only

- the defined functions via t_f and
- 1 for the function call.

©Mohammad Abdulaziz

$O(\cdot)$ is enough

$\Longrightarrow$ Reduce all additive constants to 1

Example

```
t_app (x#xs) ys = t_app xs ys + 1
```

$\Longrightarrow$ Count only

- the defined functions via t_f and
- 1 for the function call.

All other operations (variables etc) cost 0, not 1.

©Mohammad Abdulaziz

Discussion

©Mohammad Abdulaziz

Discussion

- The definition of t_f from f can be automated.

©Mohammad Abdulaziz

Discussion

- The definition of t_f from f can be automated.
- The correctness of t_f could be proved w.r.t. a semantics that counts computation steps.

©Mohammad Abdulaziz

Discussion

- The definition of t_f from f can be automated.
- The correctness of t_f could be proved w.r.t.
  a semantics that counts computation steps.
- Precise complexity bounds (as opposed to $O(\cdot)$) would require a formal model of (at least) the compiler and the hardware.

©Mohammad Abdulaziz